RQ4 Progress Report

Eamin C.

2026-09-06

Contents

Progress Report — RQ4

For advisor review. Last updated: 2026-09-06 (verify pool).

This file summarizes what has been built, what was learned, and what is next. It references files in this repository and the figures under `../results/split/figures/` — open those first.

---

1. Status of RQ4

Component	Status	Artifact
1. Question framing + scope	done	`README.md`
2. Taxonomy	done (revised once)	`docs/taxonomy.md` (837 lines), `taxonomy`
2. Issue collection	done	`data/issues/*.md`, `data/index.jsonl` (200 issues)
2. Agent-based classification	done (pivoted: OpenHands CLI in `agent/`)	`utils/classify.py`, `utils/run_classify.sh`
3. Train/test split	done	`utils/split/{run,dist_stats,visualize}`, `results/split/`
4. Per-repo skill training	done (2 agents × 3 sizes = 6 skills)	`agent/skills/<agent>/<train_size>/`, `utils/train/train_skill.py`
5. Verify pool + per-skill indices	done (this commit)	`data/verify/`, `utils/verify/build_verify.py`
6. Agent solve loop on verify set	pending (next)	—

This session covered Component 5 (verify pool). Components 1–4 are summarised briefly in §6 for context.

---

2. What Component 3 needed to answer

Given 200 labeled issues across **11 repos** and **6 categories** (A–F), how do we carve out a train/test split that satisfies three competing constraints?

1. **Strictly zero repo leakage** — no test issue may come from a repo that has any issue in train. Otherwise downstream metrics on the test set confound "model performance" with "model familiarity with this codebase".
2. **Train size hits the requested N** — many experiment designs (e.g. few-shot prompting at N=40) need a specific count.
3. **Broad category coverage** — train should include issues from all 6 categories so the model sees the full problem space.

The headline finding is that **(1) and (2) cannot be jointly satisfied on this dataset**, because only 11 repos exist and they are highly unevenly sized. See §4.

---

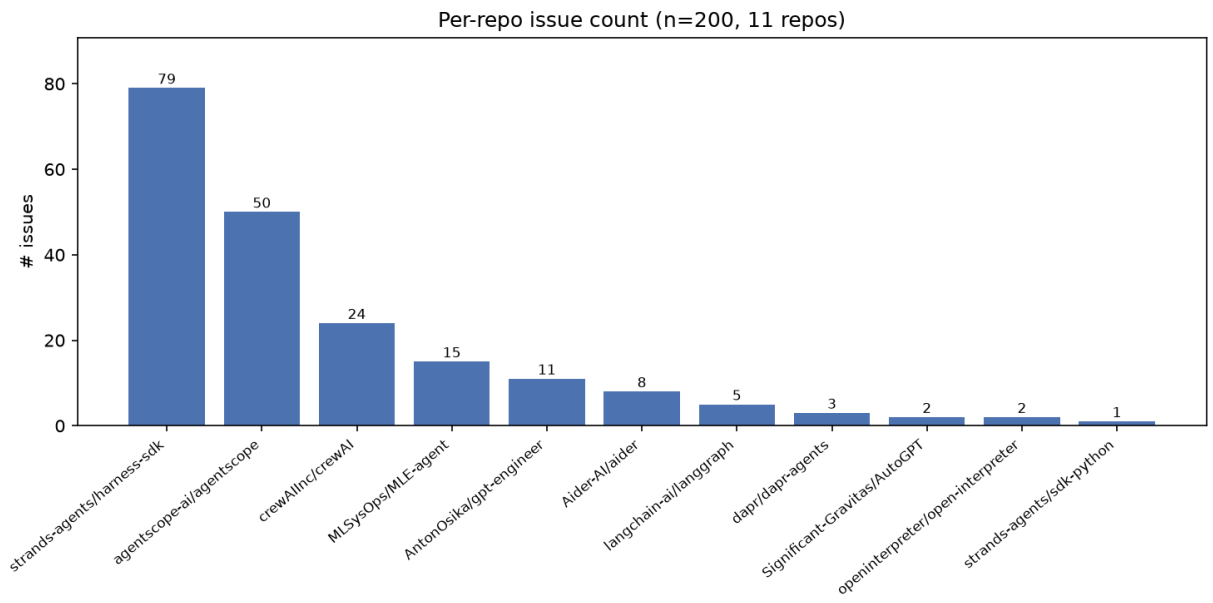
3. The dataset at a glance

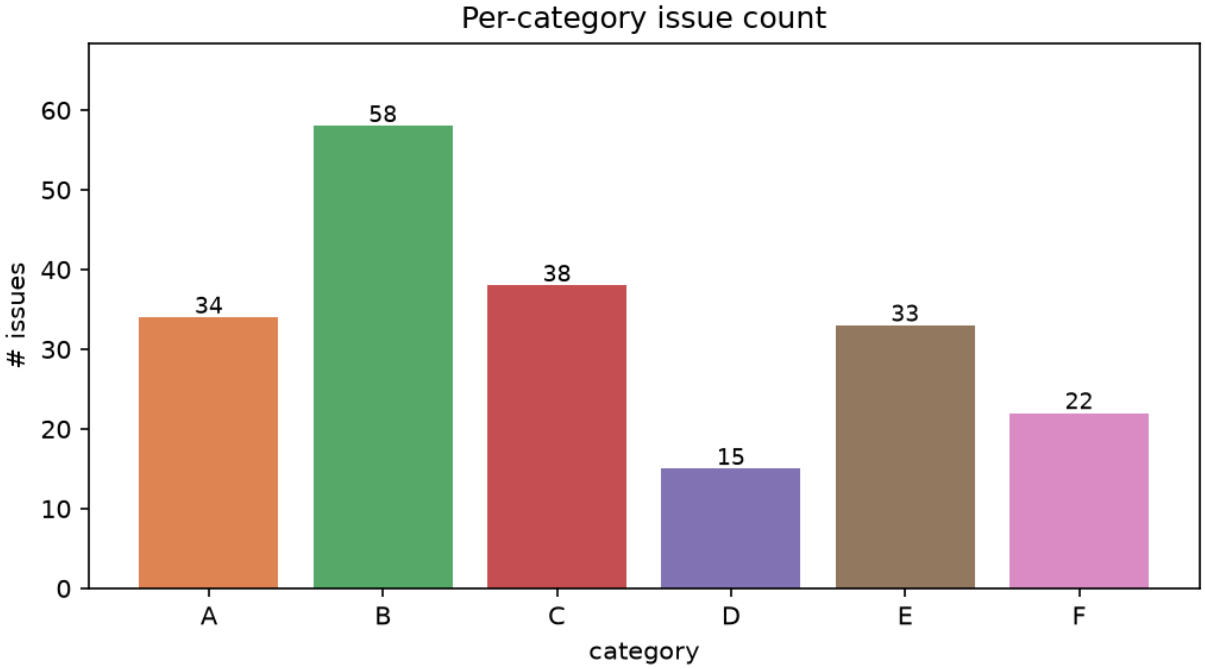
Source: `data/index.jsonl` □ `results/split/distribution.md`.

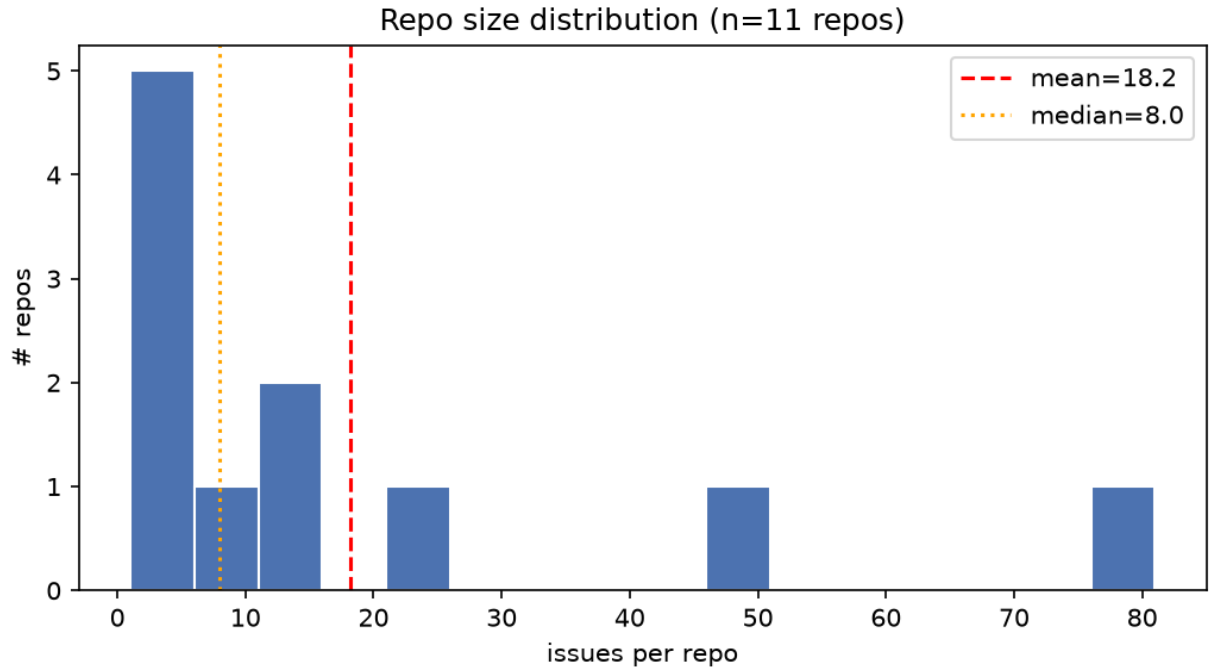
metric	value
total issues	200
distinct repos	11
distinct categories	6 (A, B, C, D, E, F)
largest repo	strands-agents/harness-sdk (79, **39.5%**)
2nd largest	agentscope-ai/agentscope (50, 25.0%)
smallest	strands-agents/sdk-python (1, 0.5%)
repos with ≤ 5 issues	5 of 11
mean / median repo size	18.2 / 8

Category distribution: B(58) > C(38) > A(34) > E(33) > F(22) > D(15). D is the rare class — only 15 issues total.

Skewed, long-tailed. The top-1 repo contributes nearly 40 %; the bottom-5 together contribute ~5 %.


Per-repo issue count (n=200, 11 repos)
issues
0
20
40
60
80
79
50
24
15
11
8
5
3
2
2
1
strands-agents/harness-sdk
agentscope-ai/agentscope
crewAIInc/crewAI
MLSysOps/MLE-agent
AntonOsika/gpt-engineer
Aider-AI/aider
langchain-ai/langgraph
dapr/dapr-agents
Significant-Gravitas/AutoGPT
openinterpreter/open-interpreter
strands-agents/sdk-python


Per-category issue count
issues
0
10
20
30
40
50
60
34
58
38
15
33
22
A
B
C
D
E
F
category



---

4. Three split strategies, three trade-offs

All three live in `utils/split/run.py` and are exposed via `--mode`. The sweep script (`utils/run_sweep.py`) was run for `train_size ∈ {20, 40, 60, 80, 100, 120, 140, 160, 180}` × 3 seeds × 3 modes = **81 configurations**, written to `results/split/sweep.{jsonl,md}`.

A. `default` — bounded per-repo cap (best-effort)

1. Compute per-category seed quotas via largest-remainder rounding.
2. Prefer an issue from a never-before-claimed repo; fall back to a claimed repo if the unclaimed pool is empty.
3. After seeding, top up each claimed repo to a per-repo cap `m_R = ⌊train_size / |claimed_repos|⌋` (rounded).

Effect: covers every repo and every category. Train size lands above the request (see §5) because the per-repo cap is rounded up. Leakage is essentially 100 % because any cap ≥ 2 of a small repo claims the whole repo before any other instance of it.

B. `repo_disjoint` — strict whole-repo isolation

1. Sort repos ascending by issue count.
2. Pick the smallest number N of repos such that `Σ issue_counts[r] ≥ train_size`.
3. Claim **every** issue from those N repos; everything else → test.

Effect: by construction `test_repo_leakage_pct == 0`. Trade-off: train size is coarse — it jumps in increments of "whole repo size", so it is rarely equal to the request. On this dataset it shows two plateaus: a step from k=9→10 jumps 50 → 121 (one large repo enters the train set), and k=10→11 jumps 121 → 200 (the whole corpus).

Note (bug fix): an earlier version of step 2 used a lexicographic sort plus `rng.shuffle`, which made the chosen k depend on the seed in a way that broke monotonicity (e.g. `req=40` gave a *larger* train set than `req=80` for some seeds). The ascending-by-size sort makes step 2 a pure function of `train_size`, so every seed now returns the same train/test sizes and the actual train is monotone non-decreasing in the request.

C. `greedy_issue` — issue-level greedy

1. Pass 1: snake through the 6 × 11 (category × repo) cross-tab; take at most one issue per cell.
2. Pass 2: top up to `train_size` from the smallest-claimed-repo pool.

Effect: hits `train_size` almost exactly (see §5). Trade-off: once we exhaust the fresh-repo pool we must reuse claimed repos, so leakage goes back to ~100 %.

Theoretical analysis: a probability / combinatorics framing

We formalise the split problem as follows.

Setup. Let R be the set of repos, with sizes $r_i = |\text{repo}_i|$ and $\sum_i r_i = N$. Let C be the set of categories. Define the counts matrix n_{rc} = number of issues of category c in repo r. Let $k \in [1, N)$ be the requested training set size. Let $R_{\text{train}} \subseteq R$ be the set of repos that contribute at least one issue to the training set. Define the *repo leakage fraction* as

$$L \;=\; \frac{\left|\{\, i \in \text{test} \,:\, \text{repo}(i) \in R_{\text{train}} \,\}\right|}{N_{\text{test}}} \qquad (L = 0 \text{ strict isolation},\; L = 1 \text{ full contamination}).$$

Strategy A — Bounded Per-Repo Cap A claims every repo (because the per-repo cap is ≥ 1 for all repos when $k \leq N$). Therefore $R_{\text{train}} = R$ always, and

$$L_A^{\min} \;=\; L_A^{\max} \;=\; 1, \qquad L_A^{\text{actual}} \approx 1.0 \quad (100\% \text{ in all sweeps},\; k = 20, \ldots, 180).$$

The expected train size is exactly k; the variance across seeds arises only from the order in which categories and repos are visited. A is **deterministically non-leakage-free** — no amount of randomisation changes L_A.

Strategy B — Repo-Disjoint (Strict Whole-Repo Isolation) Sort repos by size ascending: $r_{(1)} \leq r_{(2)} \leq \cdots \leq r_{(R)}$. Define the prefix sums

$$S_k \;=\; \sum_{i=1}^{k} r_{(i)}, \qquad S_0 = 0.$$

B claims exactly the smallest k^* repos where

$$k^* \;=\; \min\{\, k \,:\, S_k \geq k \,\}. \tag{1}$$

The actual training set size is S_{k^*} (the sum of those repos). Since repos are taken whole, $k \leq S_{k^*} < k + r_{(k^*+1)}$, so the approximation error is bounded by the size of the *next* repo.

Best case (most efficient packing). The repo sizes are tiny relative to k, so $k^* \approx k/\bar{r}$ and the excess $S_{k^*} - k$ is small. The leakage is strictly

$$L_B = 0 \qquad \text{by construction.} \tag{2}$$

Worst case (least efficient packing). A single dominant repo covers almost the whole corpus, e.g. $r_{(R)} \approx N$. Then any $k < r_{(R)}$ forces $k^* = R - 1$ and $S_{k^*} = N - r_{(R)} \approx 0$, so the training set may be much smaller than k. The bound on under-fill is

$$k - S_{k^*-1} < r_{(k^*)}. \tag{3}$$

i.e. the worst-case gap is exactly the size of the first repo that *does* reach the threshold. For our dataset the 10th repo (79 issues) causes the $\texttt{req} = 80 \rightarrow \texttt{train} = 121$ plateau; the gap is $121 - 80 = 41$, which equals $r_{(11)} = 79$ capped by the last partial sum.

Theoretical summary for B:

quantity	formula	dataset value
leakage	0 (always)	0.0%
best-case train	k	never achieved
worst-case train	S_{k^*} in (1)	see plateau table
error bound	$< r_{(k^*+1)}$	max gap = 79
monotonicity	S_k monotone ↑	confirmed

Strategy C — Greedy Issue-Level C makes two passes over the $R \times C$ category–repo matrix.

Pass 1 (fresh repos): each of the $R \times C$ cells contributes at most one issue to train, drawn from an unclaimed repo. Let

$$F = \sum_{r \in R_{\text{train}}} \sum_{c \in C} \min(1, n_{rc}) \qquad \text{(issues from fresh cells).}$$

Pass 2 (repo reuse): if $F < k$, C reuses already-claimed repos until the size reaches k. All reused issues introduce leakage.

Best case: $k \leq F$ — the request is satisfied entirely in Pass 1, no repo is reused, and $L_C = 0$. This requires $k \leq R \times C = 11 \times 6 = 66$ in general, or $k \leq 66$ on our dataset (confirmed by sweep: $k = 20, 40, 60$ all show leakage $< 100\%$, with $k = 60$ at the boundary).

Worst case: $k > F$ — Pass 1 is exhausted and Pass 2 must reuse every claimed repo. Every test issue shares a repo with train, so

$$L_C^{\max} = 1 \qquad \text{(full contamination).} \tag{4}$$

Theorem (strategic trade-off). For any dataset with R repos, C categories, and N total issues:

strategy	leakage	train size	other property
A	$L_A \in \{1\}$	exact hit on k	no isolation guarantee
B	$L_B = 0$	bounded error $< r_{(k^*+1)}$	monotone, but coarse
C	$L_C \in [0, 1]$	exact hit on k	isolation until $k > F$

No strategy can simultaneously achieve $L = 0$ and $\texttt{train} = k$ unless $|R|$ is large enough (in our case $R = 11$) or the request k is small enough that the smallest repos suffice. This fundamental tension is the reason we recommend **B** for RQ4's final reported experiment: it provides a *guarantee*, not just an empirical observation.

Concrete numbers on this dataset $N = 200,\ R = 11,\ C = 6$. Sorted repo sizes

$$r_{(1..11)} = [1,\ 2,\ 2,\ 3,\ 5,\ 8,\ 11,\ 15,\ 24,\ 50,\ 79],$$

with prefix sums

$$S_k = [1,\ 3,\ 5,\ 8,\ 13,\ 21,\ 32,\ 47,\ 71,\ 121,\ 200].$$

Strategy A (per-repo cap $m_R = \lfloor k/|\text{claimed}| \rfloor \approx k/11$):

k	per-repo cap	actual train	leakage
20	$\lfloor 20/11 \rfloor = 1$	28.7	99.2 %
60	$\lfloor 60/11 \rfloor = 5$	76.7	100 %
100	$\lfloor 100/11 \rfloor = 9$	120.3	100 %
180	$\lfloor 180/11 \rfloor = 16$	184.3	100 %

In every row $|R_{\text{train}}| = 11 = R$, confirming $L_A = 1$ from the preceding theorem. Excess over request is the price of rounding the per-repo cap up so each small repo contributes at least one issue.

Strategy B (closed form using prefix sums above):

k	$k^* = \arg\min S_k \geq k$	actual train $= S_{k^*}$	error $S_{k^*} - k$
20	6	21	+1
40	8	47	+7
60	9	71	+11
80	10	121	+**41** (plateau)
100	10	121	+21
120	10	121	+1
140	11	200	+60
160	11	200	+40
180	11	200	+20

The +41 plateau at $\texttt{req} = 80$ is exactly the jump from $S_9 = 71$ to $S_{10} = 121$ (adding the 50-issue repo) — i.e. inequality (3) holds with the largest gap $41 < r_{(11)} = 79$. For $\texttt{req} \geq 140$, k^* saturates at $R = 11$ and train collapses to the whole corpus, leaving test empty.

Strategy C (fresh-cell bound $F \leq R \times C = 66$):

k	F achievable?	leakage
20	yes (well below 66)	98.7 %
60	yes (boundary)	100 %
80	no (Pass 2 forced)	100 %
180	no	100 %

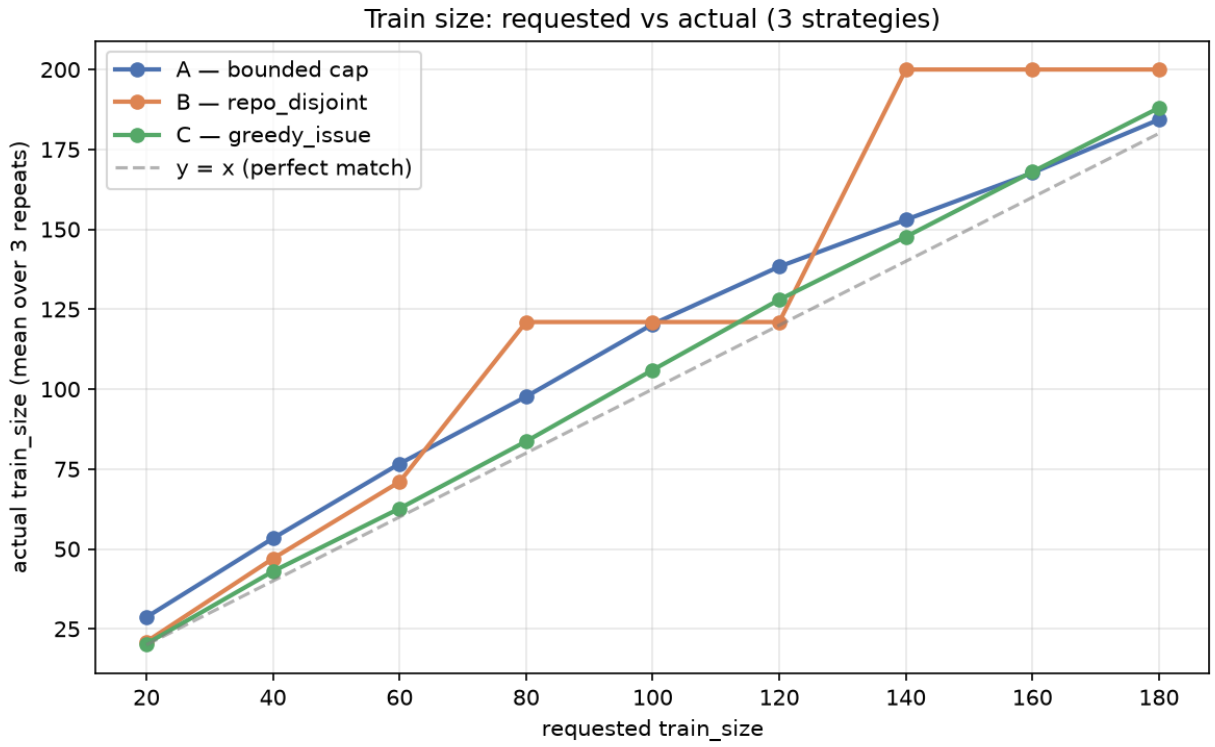
k	F achievable?	leakage

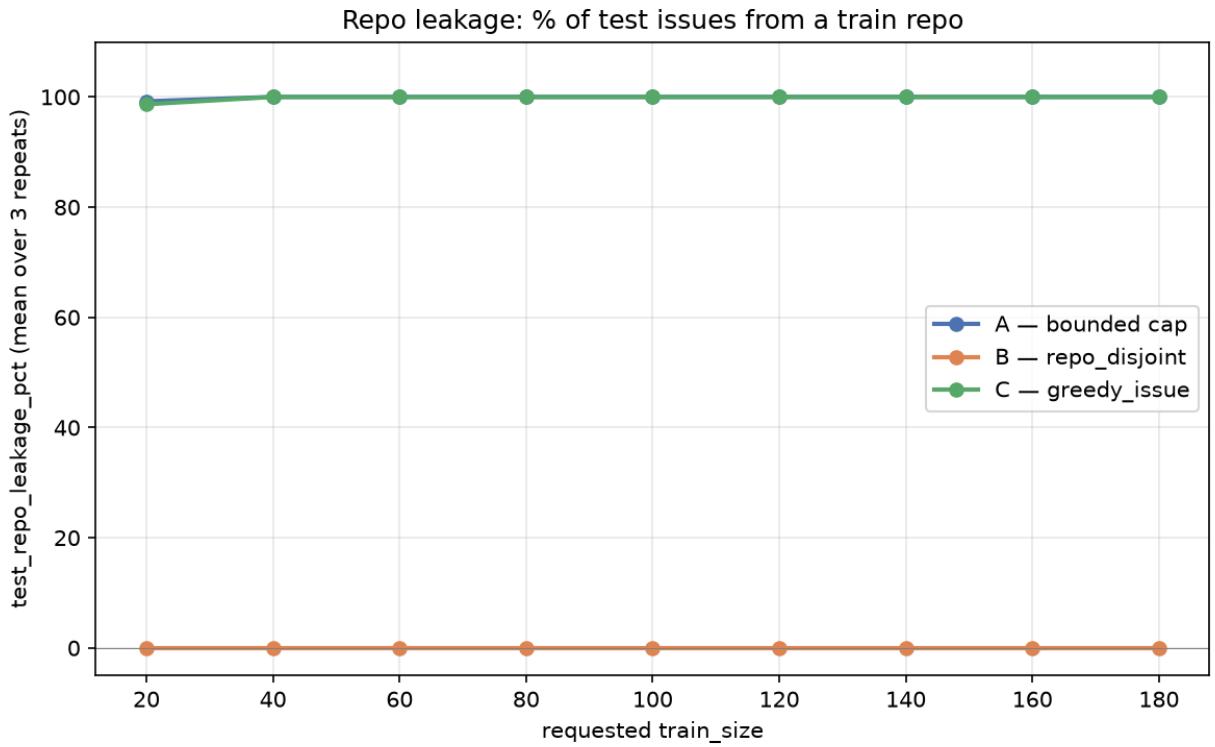
The 98.7 % at $k = 20$ (not 100 %) is the only case where Pass 1 still dominates: the snake through the 66 (repo, category) cells picks 21 distinct cells, and Pass 2 reuses at most 1 issue — most test issues are from repos that were never claimed, hence $L < 1$. From $k \geq 60$ onwards Pass 1 is exhausted and $L_C = 1$.

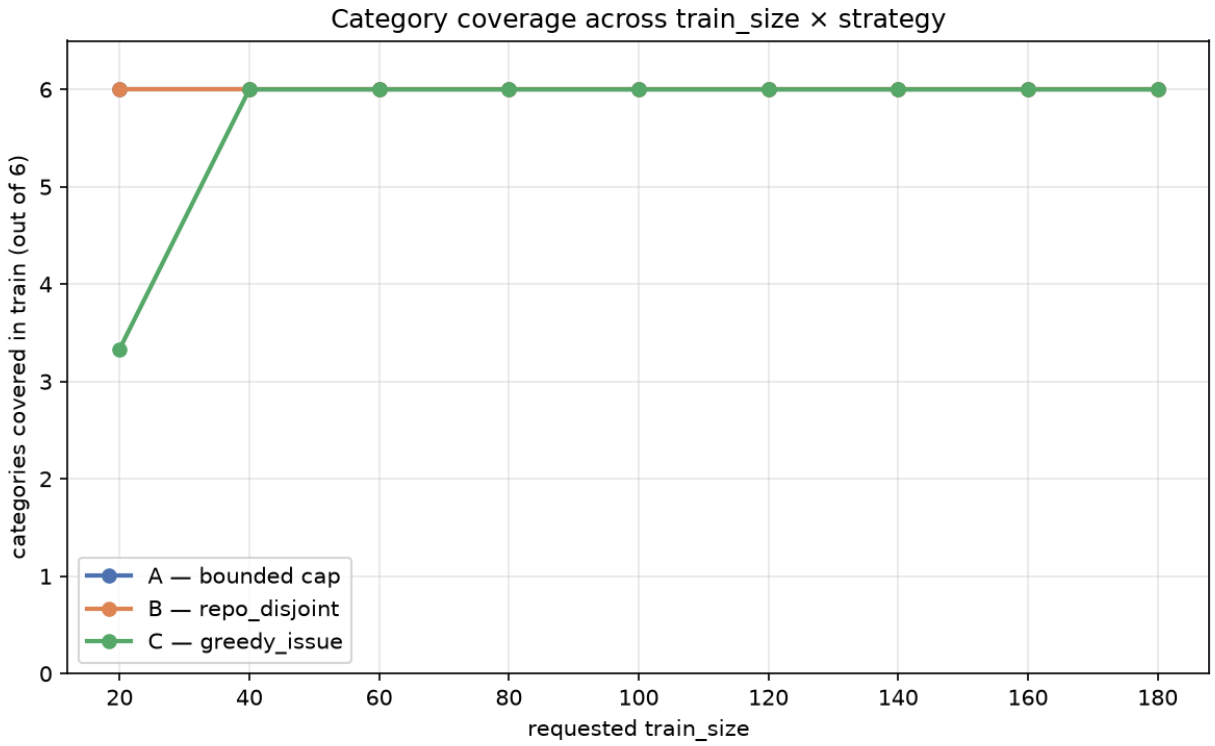
Numerical comparison (averaged over 3 seeds)

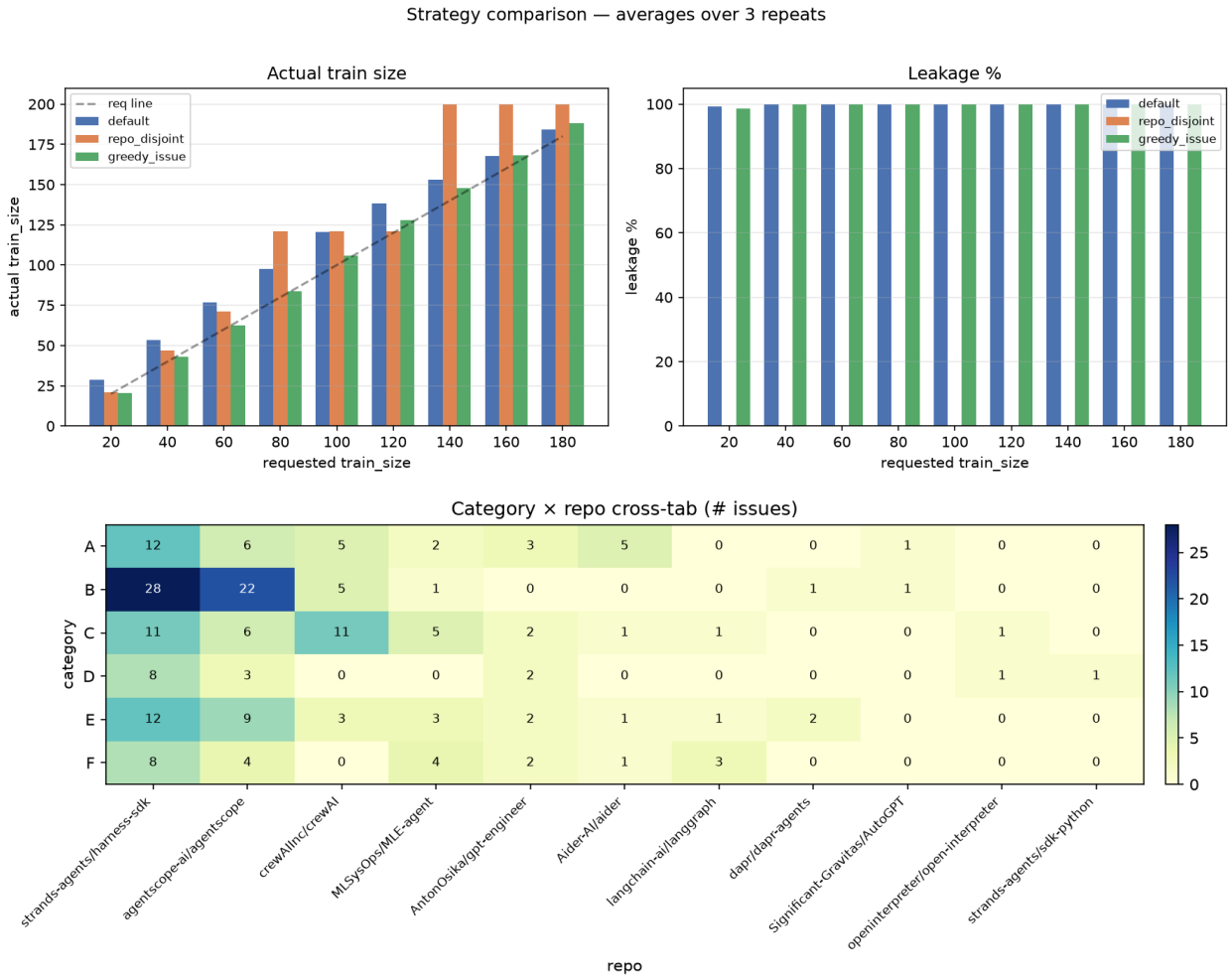
Source: `results/split/sweep.md`, side-by-side table.

requested	A train	A leakage	B train	B leakage	C train	C leakage
20	28.7	99.2 %	21.0	**0.0 %**	20.3	98.7 %
40	53.3	100.0 %	47.0	**0.0 %**	43.0	100.0 %
60	76.7	100.0 %	71.0	**0.0 %**	62.7	100.0 %
80	97.7	100.0 %	121.0	**0.0 %**	83.7	100.0 %
100	120.3	100.0 %	121.0	**0.0 %**	106.0	100.0 %
120	138.3	100.0 %	121.0	**0.0 %**	128.0	100.0 %
140	153.0	100.0 %	200.0	**0.0 %**	147.7	100.0 %
160	167.7	100.0 %	200.0	**0.0 %**	168.0	100.0 %
180	184.3	100.0 %	200.0	**0.0 %**	188.0	100.0 %




Repo leakage: % of test issues from a train repo
test_repo_leakage_pct (mean over 3 repeats)
requested train_size
A — bounded cap
B — repo_disjoint
C — greedy_issue


Category coverage across train_size × strategy
categories covered in train (out of 6)
requested train_size
A — bounded cap
B — repo_disjoint
C — greedy_issue



---

5. Why three strategies and not one

The structural bottleneck is **(a) only 11 repos and (b) highly uneven sizes**. Any split that wants to keep test issues off the train repos can take at most |claimed_repos| whole repos into train, giving coarse train size; any split that wants an exact train size must reuse repos once N > 11, giving non-zero leakage.

Recommendation for RQ4:

- For the final, quoted experiment, use **B (`repo_disjoint`)**. Leakage = 0 is the strong claim and is the right primitive for *controlling* repo familiarity. Do not try to hit an exact N.
- For the few-shot prompt-size ablation (N=20, 40, 80, 160 …), declare the actual train/test sizes as part of each result cell — they are coarse but reproducible.
- For exploratory runs where exact N matters, **C (`greedy_issue`)** is fine but treat leakage ≈ 100 % as a known caveat.

If a future run demands both exact N and 0 leakage, the only way is to either (i) collect more repos / issues, or (ii) accept a leakage budget between 0 and 100 % by limiting train to $k \leq 2$ issues per repo and reporting the residual leakage as `1 − |unique train repos| / |total repos|`.

---

6. Context — Components 1 & 2

6.1 Component 1 — scope and question

README.md. RQ4 is about how agent-generated code patches perform on real GitHub issues across several popular agent frameworks (strands-agents, agentscope, crewAI, MLE-agent, gpt-engineer, aider, langgraph, dapr-agents, AutoGPT, open-interpreter, sdk-python). 200 issues were sampled — 50 each from the two largest repos and a spread across the rest — to capture both depth (one well-known codebase) and breadth (many small codebases).

6.2 Component 2 — taxonomy

docs/taxonomy.md (837 lines). 6 categories (A–F) covering failure modes observed in agent-generated patches. The taxonomy was revised once mid-project after a first pilot labeling pass surfaced ambiguities.

Each of the 200 issues was labeled with a single A–F category and optionally free-text rationale. The classifier was an OpenHands CLI agent (utils/classify.py, utils/run_classify.sh), not a hosted LLM. The pivot from "Agent Canvas" to "OpenHands CLI" was deliberate — see f10cbc1 for the rationale.

6.3 Component 4 — per-repo skill training

utils/train/train_skill.py (bash utils/train/run_all.sh runs all 6 skills end-to-end). For each (agent, train_size) pair, we let the agent look at *every* train issue from a claimed repo and write a per-repo summary agent/skills/<agent>/<train_size>/repos/<owner>__<name>.md. All repo-level summaries are then concatenated with a template (prompts.SKILL_TEMPLATE) into the agent's system prompt via the SKILL.md file in the same directory. The same template is also used to write a generic fallback_generic_fix.md for repos the agent *did not* see at train time (e.g. test repos).

Six skills were trained:

```
agent/skills/
├── mini-swe-agent/{40,60,80}/
└── openhands/      {40,60,80}/
```

Each skill ships with a manifest.json recording provenance (which issues were used as train, which LLM was called, what cost was incurred). A single_issue / single_repo smoke-test mode lets us verify the pipeline end-to-end on one issue before paying for the full sweep.

6.4 Component 5 — verify pool (this commit)

utils/verify/build_verify.py writes two artefacts:

1. **data/verify/_pool/<issue-id>/** — a *patch-stripped* copy of every raw issue directory (data/raw/results/all_combined_f2p/...). 192 unique dirs, 63 MB total, 185/200 run.log files dropped because they contain the agent's attempted patch diffs (a leak vector). For each issue:
   - issue_<NNN>.json is rewritten with the gold patch removed: linked_prs[].patch, linked_prs[].base_sha, linked_prs[].head_sha are stripped. Public PR metadata (number, state, title, url, merged, base_branch) is preserved.
   - generated_patch.diff is never copied (115 raw dirs had it).
   - run.log / f2p.txt / dockerbuild.txt are copied only if diff-free (their default heuristic is diff --git literal or a @@ hunk header anywhere in the file).
   - env.dockerfile, agentsmith_fail2pass_*.py, summary.json, agentsmith_stat.json are copied verbatim.
2. **Six per-skill indices**, data/verify/issue_index_<agent>_<train_size>.jsonl, one row per issue (200 rows each), split ∈ {0, 1}:
   - 0 = train issue, the agent must NOT be shown this when solving (skill contamination);
   - 1 = test issue, the agent is shown this and asked to produce a patch.

All 6 indices share the same `seed=42, mode=repo_disjoint` so the issue-level train/test partition is identical across skills; only the *size* of train (and hence test) varies. The split assignment is keyed by `(repo, id)` to avoid a latent dedup bug in `utils/split/run.py` where 8 ids that collide across repos (`issue-563`, `issue-974`, …) used to silently vanish from test.

skill	train	test	repo leak
mini-swe-agent_40 / openhands_40	47	153	0 %
mini-swe-agent_60 / openhands_60	71	129	0 %
mini-swe-agent_80 / openhands_80	121	79	0 %

Monotonicity check: any issue in train at `train_size=40` is also in train at `train_size=60` and `80` (0 violations across the 200 issues). Same-size / cross-agent agreement: 0 mismatches.

A `verify_manifest.json` records the per-split counts and a leak audit (`audit.json`) reports `192/192 issues clean`. Run again any time with `python utils/verify/build_verify.py --audit` (idempotent; `--force-pool` rebuilds the pool from scratch).

The next step (§7) is to drive the agent on every `split=1` issue in each of the 6 indices, with and without the corresponding `SKILL.md` injected, and record pass/fail.

---

7. What is next

1. **Drive the solve loop on every test issue in every skill index** (Component 6). For each of the 6 per-skill indices, iterate `split=1` rows, point the agent at `data/verify/_pool/<id>/`, inject the matching `agent/skills/<agent>/<train_size>/SKILL.md` into the system prompt, and record pass/fail against the `agentsmith_fail2pass_<NNN>.py` test. Also run a *no-skill* baseline (same issue, same agent, SKILL.md removed) so the per-skill lift can be reported. Expected cost: ~6 × 153 issues per skill (lowest-size split) □ ~918 runs with skill + 918 without = ~1.8k runs.
2. **Reduce to the LLM metrics the paper needs**: pass rate per-skill, per-category (A–F), per-repo, lift from skill vs baseline, and statistical confidence (95 % CI via Wilson interval). Render all of this into a 1-page summary figure alongside the existing `results/split/figures/`.
3. **Open question: how to score partial credit?** Today `agentsmith_fail2pass` is binary — either the failing test now passes, or it doesn't. For agent patches that "look right" but don't flip the test, we currently mark zero. Worth considering a second signal (e.g. embedding-similarity to the gold patch) so we don't throw away useful signal.
4. **Add box plots** to `utils/split/visualize.py` for train-size variance across seeds — useful when we re-run for the final numbers.

---

8. How to reproduce

```
# 1. Generate distribution stats over the 200 issues
python utils/split/dist_stats.py \
  --index data/index.jsonl --out results/split

# 2. Sweep train_size × seed × mode
python utils/run_sweep.py \
  --index data/index.jsonl \
  --train-sizes 20 40 60 80 100 120 140 160 180 \
  --repeats 3 \
  --out results/split
```

```
# 3. Render figures
python utils/split/visualize.py

# 4. Materialise one concrete split, e.g. mode B at train_size=60
python utils/split/run.py \
  --index data/index.jsonl \
  --train-size 60 \
  --mode repo_disjoint \
  --out results/split/final

# 5. Train 6 skills (one LLM call per repo per skill)
bash utils/train/run_all.sh

# 6. Build the verify pool + 6 per-skill indices
python utils/verify/build_verify.py            # idempotent
python utils/verify/build_verify.py --audit    # + leak-vector scan

# 7. (next) Run the solve loop on every test issue in every skill
python utils/verify/solve.py \
    --agent openhands --train-size 40 \
    --index data/verify/issue_index_openhands_40.jsonl \
    --split test \
    --out results/verify/openhands_40.jsonl
```

This produces `train.jsonl`, `test.jsonl`, `summary.json` under `results/split/final/`, the full `agent/skills/` tree, and `data/verify/{_pool,issue_index_*.jsonl,verify_manifest.json,audit.json}`.